

Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology
Arts Sciences
TH Köln

Lernraum IV: Entwurf von Algorithmen

- Sortieralgorithmen als Beispiele für den Entwurf von Algorithmen
  - Selectionsort
  - Insertionsort
  - Heapsort
  - Mergesort
  - Quicksort
- Dynamische Programmierung
- ~~P- vs. NP-Vollständigkeit~~
- ~~Caching, B-Baum~~

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können für eine gegebene Sortieraufgabe einen geeigneten Sortieralgorithmus auswählen und implementieren, indem sie einen Algorithmus aus der Menge

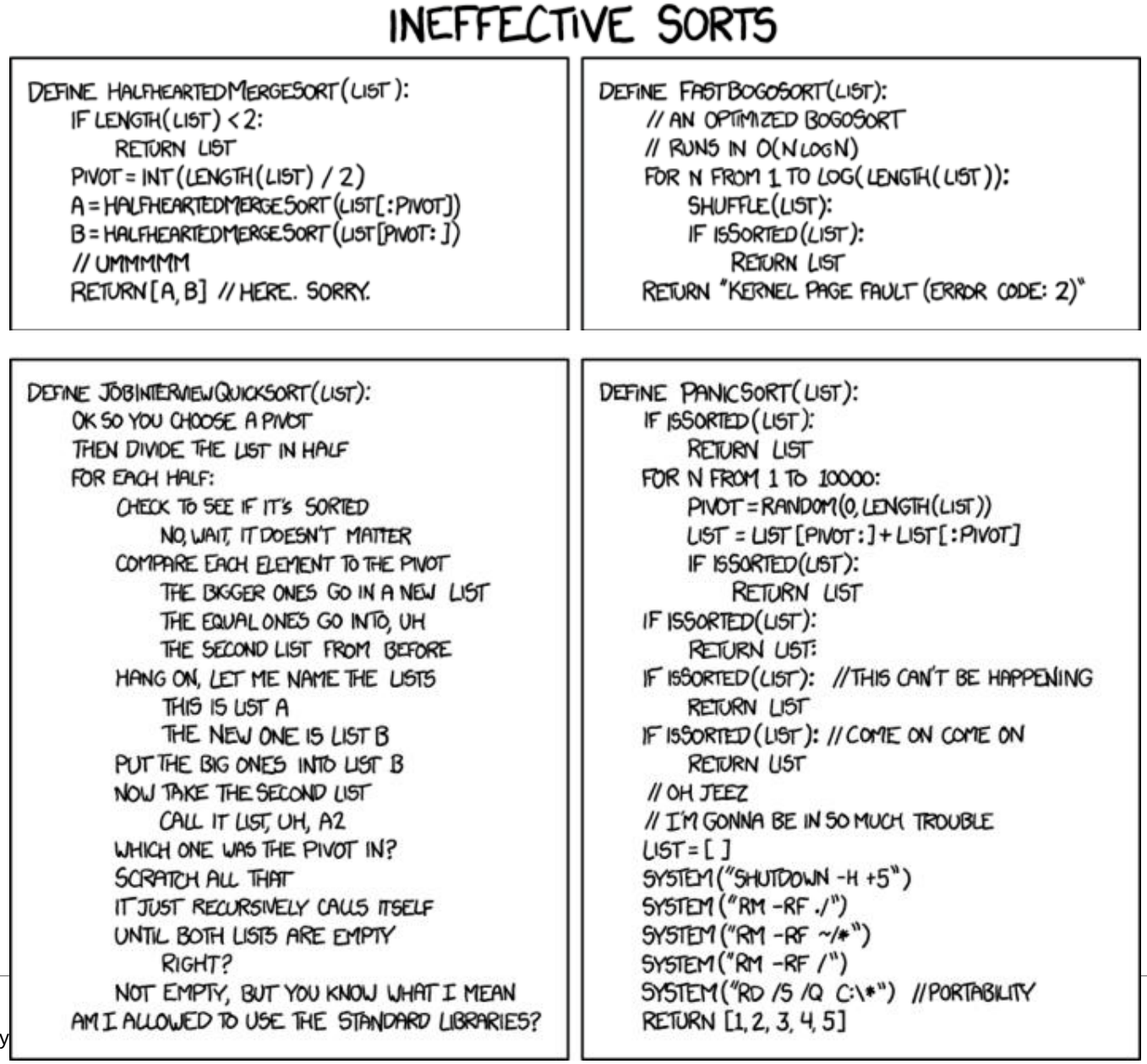
- Selectionsort
- Insertionsort
- Heapsort
- Mergesort
- Quicksort
- Bucketsort

auswählen, seine Eigenschaften wie Laufzeit, Speicherbedarf und Stabilität bei der Wahl beachten und den gewählten Algorithmus implementieren, um später praktische Problemstellungen, die eine Sortierung erfordern, lösen zu können.

s. Übungsblatt 9

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Sortieralgorithmen



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 19 WI - KASEY CHAMPION

Wo befinden wir uns im Modul?

Dieses Modul ist „Datenstrukturen und Algorithmen“ (aka Algorithmik).

Datenstrukturen organisieren unsere Daten, damit wir sie effektiv verarbeiten können.

Algorithmen verarbeiten unsere Daten tatsächlich!

Wir fangen an, uns auf Algorithmen zu konzentrieren.

Wir beginnen mit dem Sortieren

- Ein sehr verbreiteter, allgemein nützlicher Vorverarbeitungsschritt
- Und ein geeigneter Ansatz, um ein paar verschiedene Ideen für den Entwurf von Algorithmen zu diskutieren.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Anwendungen für Sortieralgorithmen

- Grafikengine:
  - Objekte für Rendering sortieren, z.B. nach Tiefe / Transparenz
- Datenbanken & Suche
  - Datensätze nach Schlüssel sortieren
  - Beispiele: Kunden nach Name, Produkte nach Preis
  - → Binäre Suche setzt sortierte Daten voraus!
- Datenanalyse & Logs
  - Zeitreihen / Logeinträge chronologisch sortieren

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Arten von Sortieralgorithmen

Vergleichsbasierte Sortierstrategien

- paarweiser Vergleich der zu sortierenden Elemente
- Allgemeine Verfahren, funktionieren für die meisten Datentypen
- Bei Laufzeitanalyse wird davon ausgegangen, dass Vergleich zweier Elemente in O(1) erfolgt

Nicht-vergleichsbasiertes Sortieren

- Nutzung spezifischer Eigenschaften der Elemente in der Liste, um schnellere Laufzeiten zu erzielen
- Laufzeit typischerweise O(n)
- Zum Beispiel: Sortieren von kleinen Ganzzahlen oder kurze Zeichenketten

- In diesem Modul konzentrieren wir uns auf Vergleichsbasiertes Sortieren

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 18 SP - KASEY CHAMPION

Eigenschaften von Sortieralgorithmen

In-place

- Ein Sortieralgorithmus ist „in-place“, wenn er nur O(1) zusätzlichen Speicher alloziert.
- Modifiziert Eingabe-Array (kopiert Daten nicht in ein neues Array)
- Nützlich, um die Speichernutzung zu minimieren

Laufzeit

- Wir wollen natürlich, dass unsere Algorithmen schnell sind.
- Das Sortieren ist so alltäglich, dass wir oft anfangen, uns Gedanken über konstante Faktoren zu machen.

Eigenschaften von Sortieralgorithmen

Stabilität

Ein Sortieralgorithmus ist stabil, wenn alle gleichwertigen (bezüglich eines Merkmals) Elemente vor und nach der Sortierung in der gleichen relativen Reihenfolge bleiben.

Warum ist das wichtig?

- Sortieren nach mehreren Merkmalen: Dabei wird Gleichheit bei einem Merkmal mit sekundären Merkmalen aufgelöst.

Eingabe-Array:	[**(8, “fox”, a)**, (9, “dog”), (4, “cow”), **(8, “beaver”, c)**]	
Algorithmus 1:	[(4, “cow”), **(8, “fox”, a)**, **(8, “beaver”, c)**, (9, “dog”)]	Stabil
Algorithmus 2:	[(4, “cow”), **(8, “beaver”, c)**, **(8, “fox”, a)**, (9, “dog”)]	Instabil

Eigenschaften von Sortieralgorithmen - Stabilität

Beispiel

Sortieren aller Studierender nach Geburtsjahr. Bei identischem Geburtsjahr, soll aufsteigend nach Nachname sortiert werden.

Eingabe-Array:	[**(2000, “Müller”)**, (2001, “Meier”), (1999, “Wolf”), **(2000, “Fröhlich”)**]
Schritt 1 (Nachname):	[**(2000, “Fröhlich”)**, (2001, “Meier”), **(2000, “Müller”)**, (1999, “Wolf”)]
Schritt 2 (Jahr):	[(1999, “Wolf”), **(2000, “Fröhlich”)**, **(2000, “Müller”)**, (2001, “Meier”)]

Schritt 2 geht nur mit einem stabilen Sortieralgorithmus.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Es gibt sehr viele Sortieralgorithmen

Quicksort, Merge sort, in-place merge sort, heap sort, insertion sort, intro sort, selection sort, timsort, cubesort, shell sort, bubble sort, binary tree sort, cycle sort, library sort, patience sorting, smoothsort, strand sort, tournament sort, cocktail sort, comb sort, gnome sort, block sort, stackoverflow sort, odd-even sort, pigeonhole sort, bucket sort, counting sort, radix sort, spreadsort, burstsort, flashsort, postman sort, bead sort, simple pancake sort, spaghetti sort, sorting network, bitonic sort, bogosort, stooge sort, insertion sort, slow sort, rainbow sort…

Algorithmen Entwurfsmuster

Algorithmen entstehen nicht einfach aus dem Nichts.

Es gibt gängige Muster, die wir für die Entwicklung neuer Algorithmen verwenden.

Viele von ihnen sind auf das Sortieren anwendbar (weitere Muster werden wir nächste Woche kennenlernen)

Iterative Verbesserung von Invarianten

- Schritt für Schritt einen weiteren Teil der Eingabe zur gewünschten Ausgabe machen.

Verwendung von Datenstrukturen

- Beschleunigung unserer bestehenden Algorithmen

Teile und beherrsche (divide and conquer)

- Aufteilen der Eingabe
- Lösen der einzelnen Teile (rekursiv)
- Kombinieren der gelösten Teile zu der Lösung

Algorithmen Entwurfsmuster 1

Iterative Verbesserung von Invarianten

- Schritt für Schritt einen weiteren Teil der Eingabe zur gewünschten Ausgabe machen.

Wir werden iterative Algorithmen schreiben, die die folgende Invariante erfüllen:

- Nach k Iterationen der Schleife werden die ersten k Elemente des Arrays sortiert sein.

Beispiele:

- Insertionsort
- Selectionsort

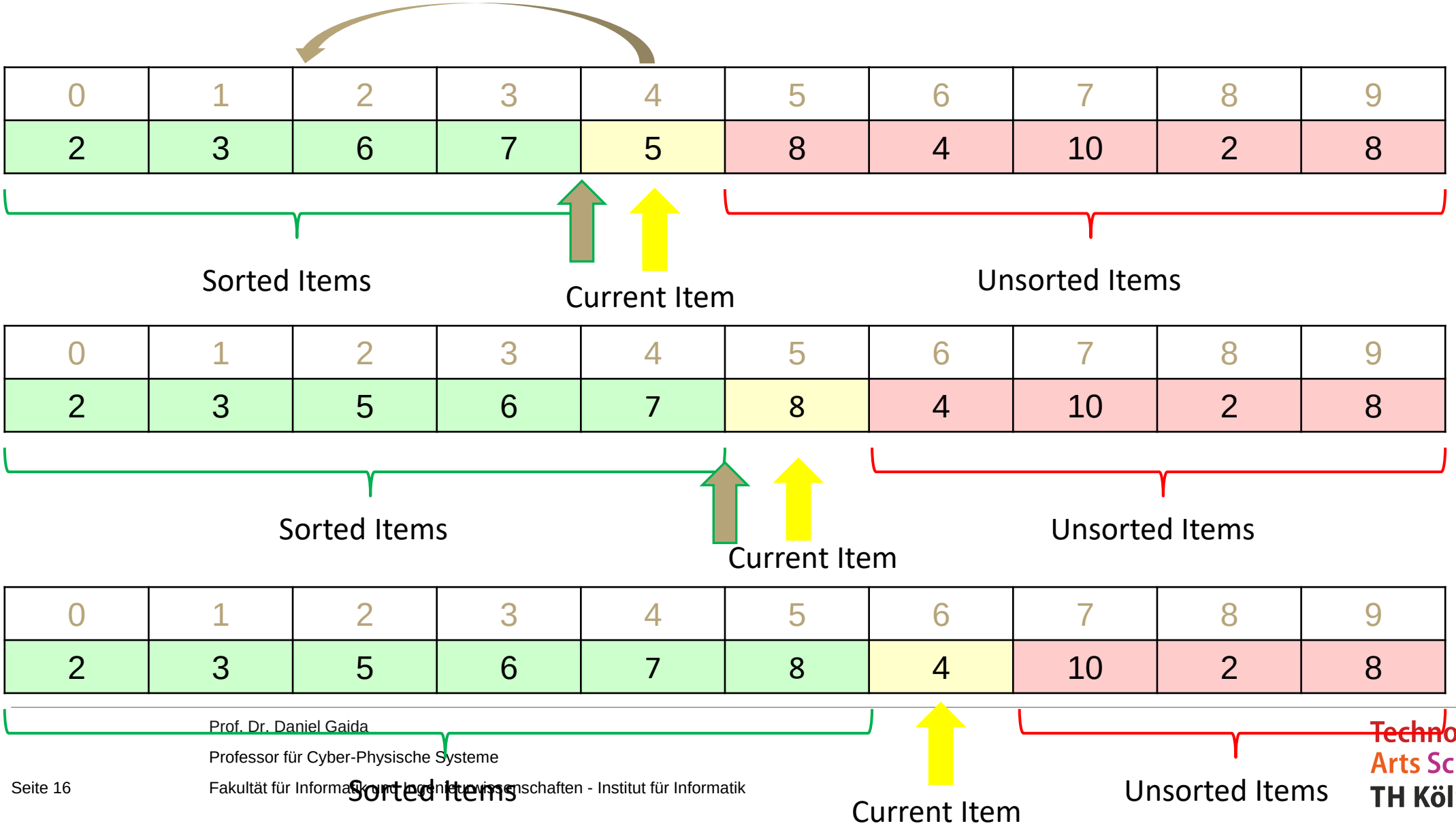
Anderes Beispiel: Dijkstras Algorithmus

- Nach k Iterationen haben wir die k nächsten Nachbarn des Startknoten gefunden

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Insertionsort (Sortieren durch Einfügen)

https://www.youtube.com/watch?v=ROalU379l3U

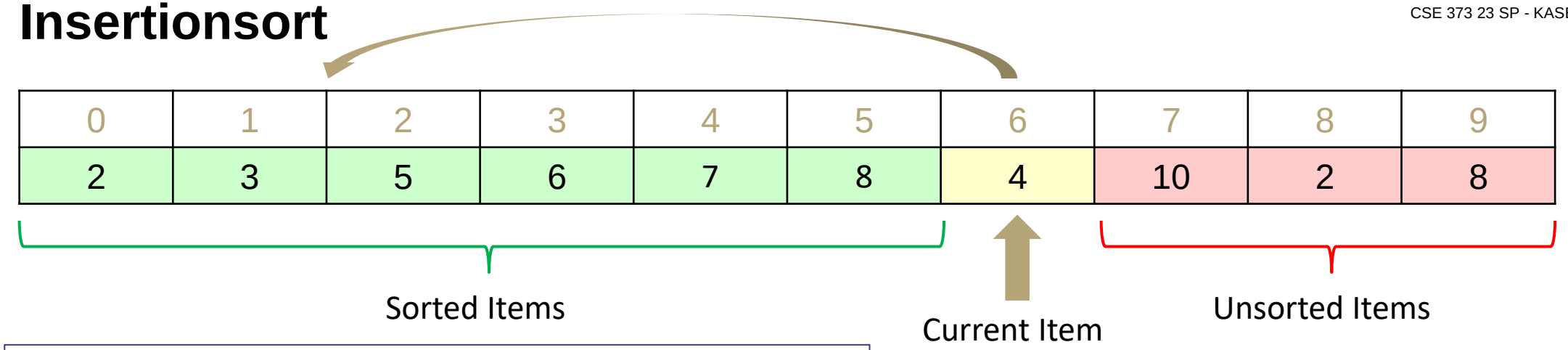


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 23 SP - KASEY CHAMPION

Insertionsort

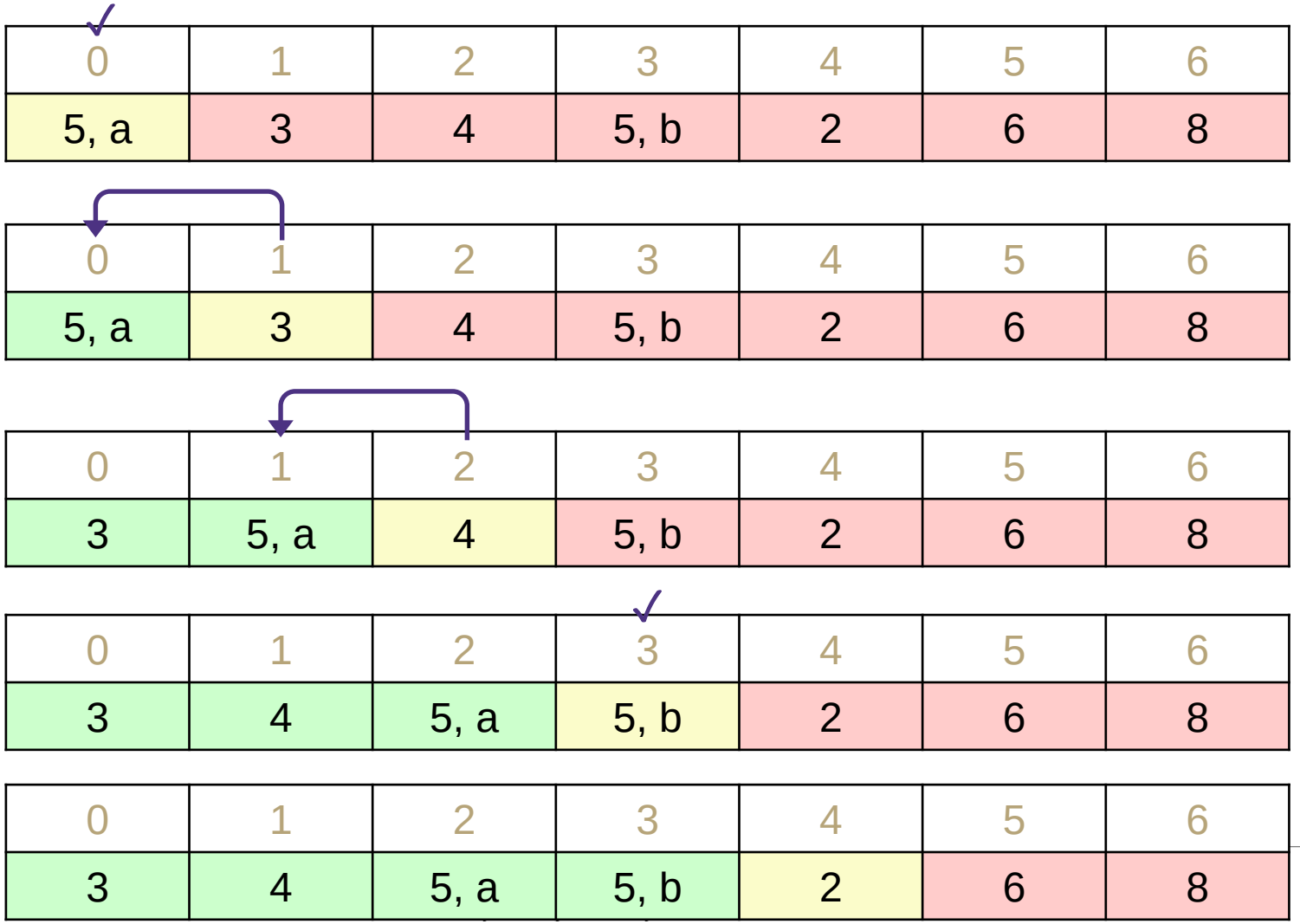



```
void insertionSort(list) {
   for (currentItem in entire list)
      if(currentItem is smaller than largestSorted)
         int newIndex = findSpot(currentItem)
         shift(list, newIndex, currentItem)
}
int findSpot(currentItem) {
   for (sorted list going backwards)
      if (spot found) return spot
}
void shift(list, newIndex, currentItem) {
 for (i = index of currentItem; i >= newIndex; i--)
    list[i+1] = list[i]
 list[newIndex] = currentItem
}
```

Worst case runtime?	$\Theta(n^2)$
Best case runtime?	$\Theta(n)$
In-practice runtime?	$\Theta(n^2)$
Stable?	Yes
In-place?	Yes
Useful for:	Mostly sorted collections of primitives

$$\sum_{i=0}^{n-1} \sum_{j=i-1}^{0} c$$

Insertionsort Stabilität



Insertionsort ist stabil.

- Alle Tauschvorgänge finden zwischen benachbarten Elementen statt, um das aktuelle Element an die korrekte relative Position innerhalb des sortierten Teils des Arrays zu bringen
- Duplikate werden immer in ihrer ursprünglichen Anordnung miteinander verglichen, so dass die Sortierung beibehalten werden kann.

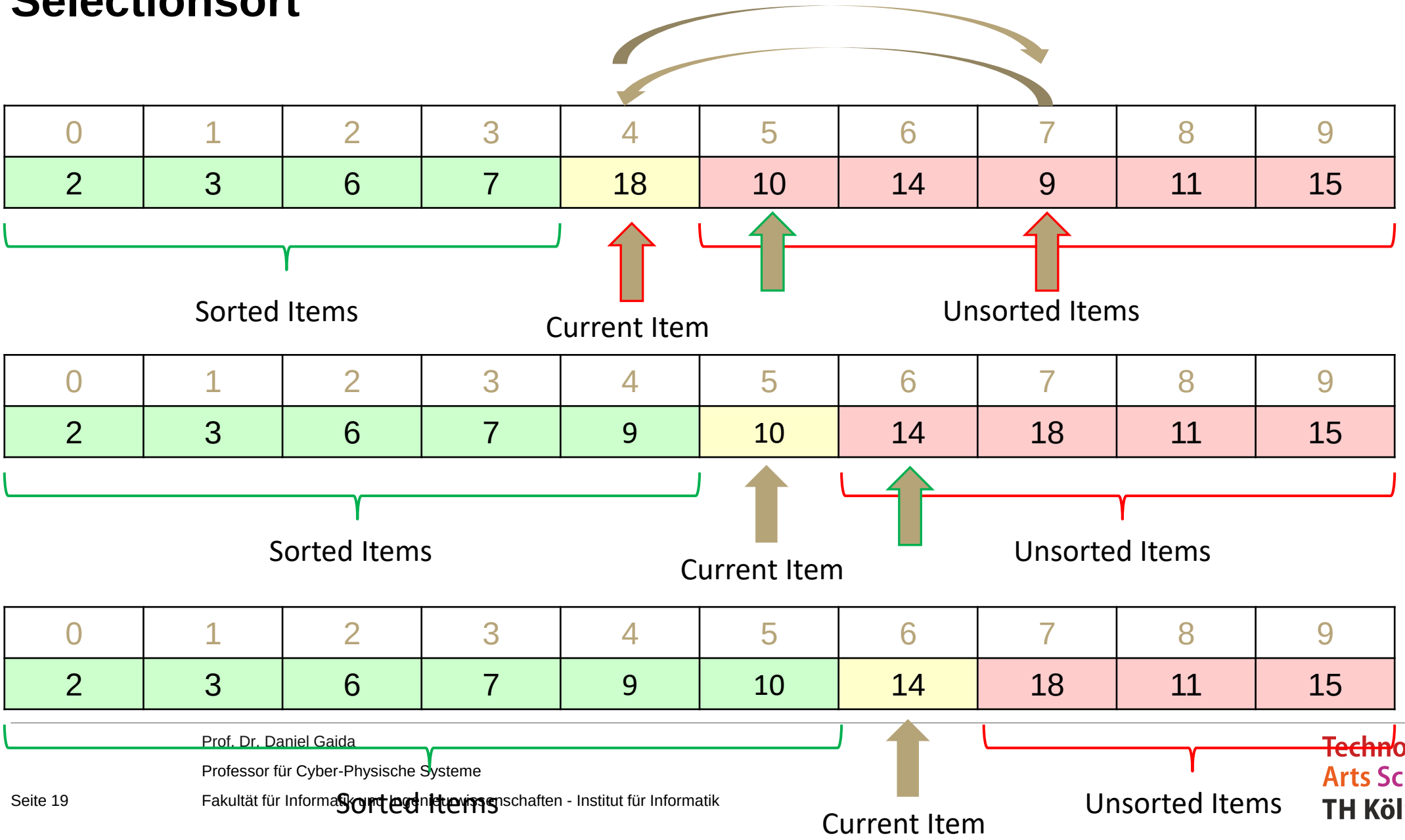
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SP – CHAMPION & CHUN

https://www.youtube.com/watch?v=Ns4TPTC8whw

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Selectionsort

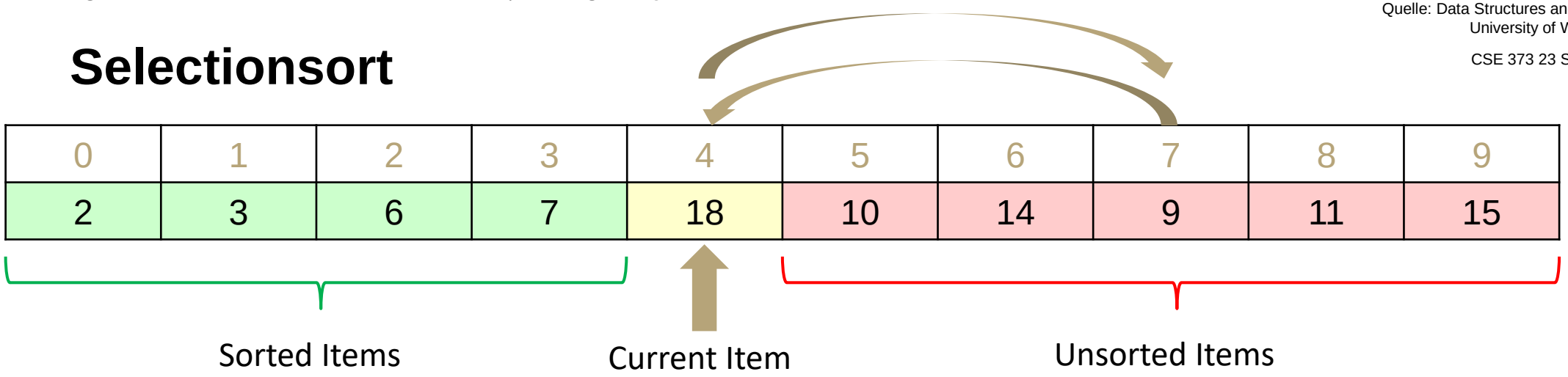


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 23 SP - KASEY CHAMPION

Selectionsort



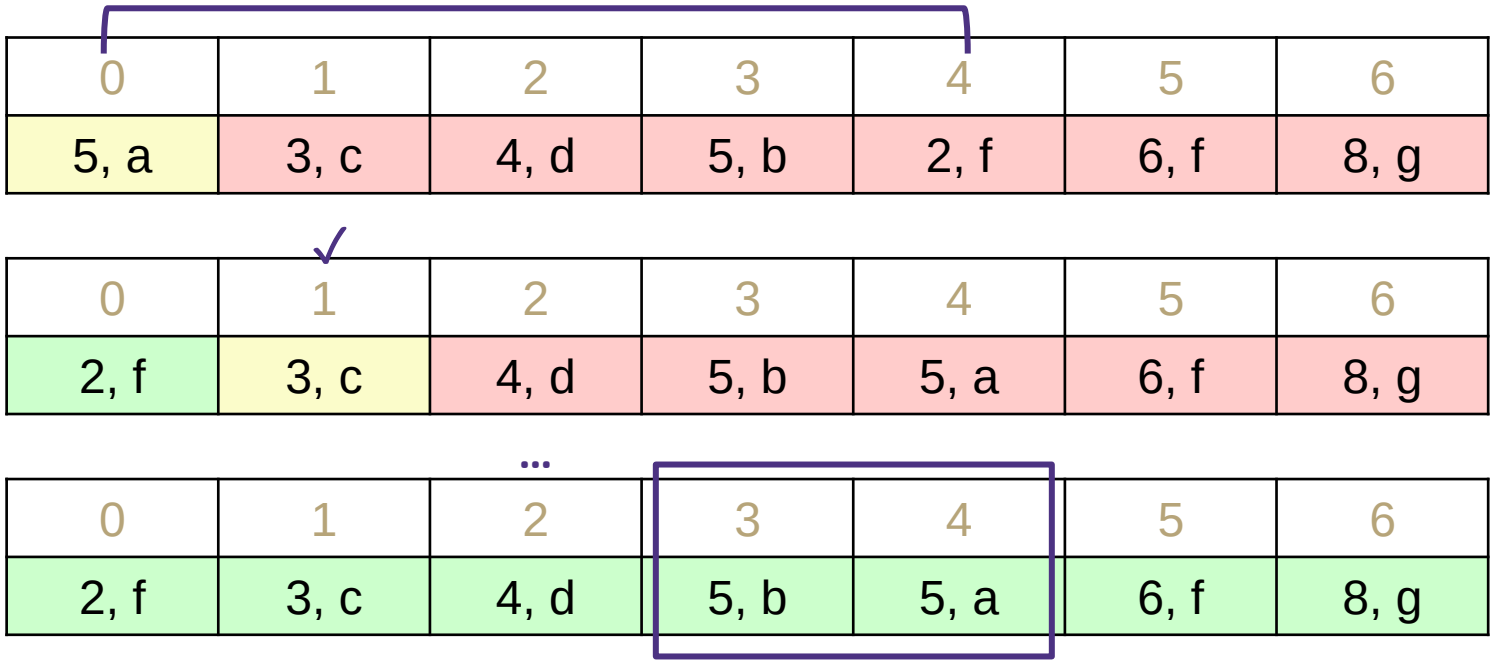

```
void selectionSort(list) {
   for (currentItem in entire list)
      int newIndex = findNextMin(currentItem)
      swap(newIndex, currentItem)
}
int findNextMin(currentItem) {
   min = currentItem
   for (item in unsorted list)
      if (list[item] < list[min])
         min = item
   return min
}
void swap(newIndex, currentItem) {
   temp = list[currentItem]
   list[currentItem] = list[newIndex]
   list[newIndex] = temp
}
```

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} c$$

Worst case runtime?	$\Theta(n^2)$
Best case runtime?	$\Theta(n^2)$
In-practice runtime?	$\Theta(n^2)$
Stable?	No
In-place?	Yes
Useful for:	Top k sort without needing extra space

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Selectionsort: Stabilität



Das Vertauschen nicht benachbarter Elemente kann zur Instabilität von Sortieralgorithmen führen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP – CHAMPION & CHUN

Algorithmen Entwurfsmuster 2

Verwendung von Datenstrukturen

- Beschleunigung unserer bestehenden Algorithmen
- Bspw. wie bei Algorithmus von Kruskal

Selectionsort:

- Sucht kleinstes Element in unsortiertem Teil der Liste und tauscht es mit currentItem
- Laufzeit immer in $\Theta(n^2)$

Wenn wir nur eine Datenstruktur hätten, die gut darin wäre, das kleinste verbleibende Element im unsortierten Teil der Liste zu finden ...

- Wir haben sie!

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Heapsort

1. Führe Floyd's Heap Construction Algorithmus auf den Daten aus
2. Rufe removeMin() n-Mal auf

```
public E[] heapSort(input) {
   E[] heap = buildHeap(input)
   E[] output = new E[n]
   for (i=0; i < n; i++)
      output[i] = removeMin(heap)

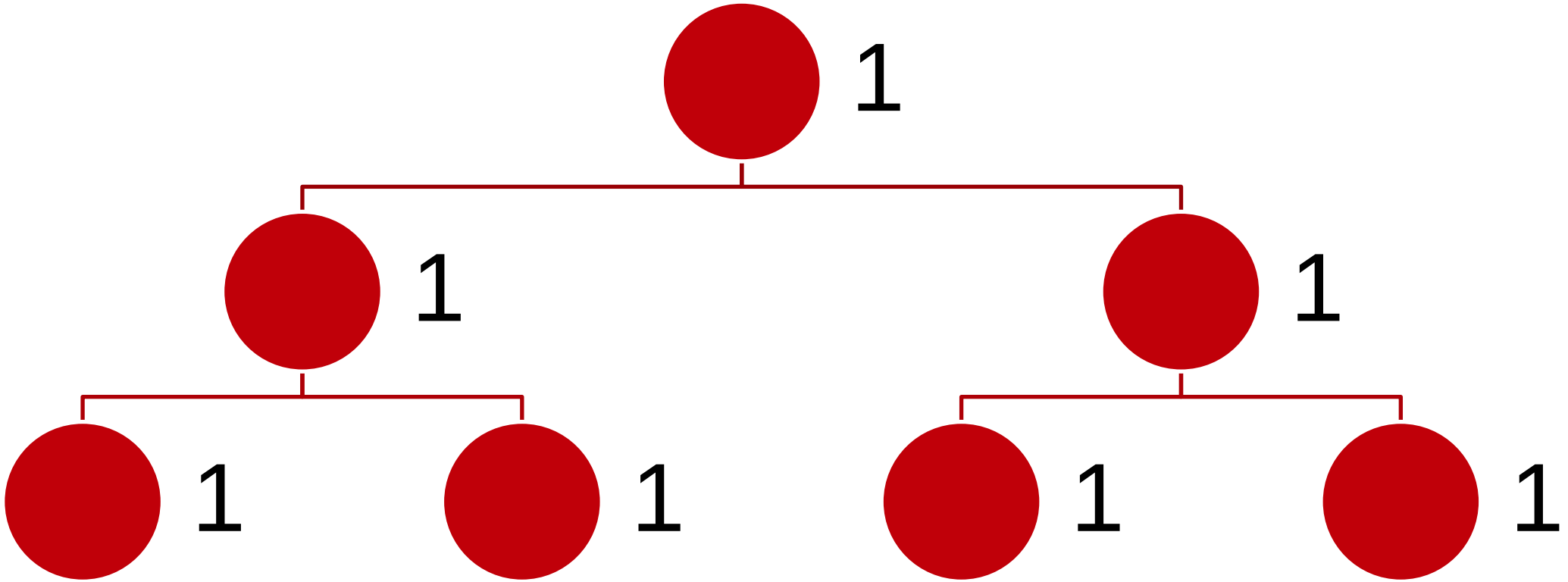
   return output
}
```

Worst case runtime?	$\Theta(n \log n)$
Best case runtime?	$\Theta(n)$
In-practice runtime?	$\Theta(n \log n)$
Stable?	No
In-place?	If we get clever…

Visualisierung:

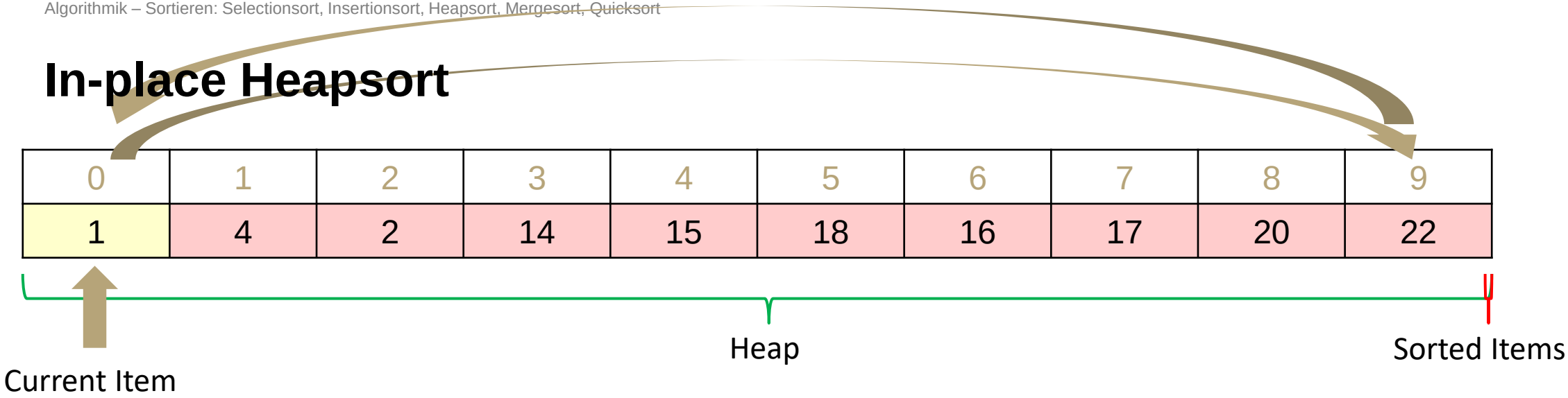
https://www.youtube.com/watch?v=Xw2D9aJRBY4

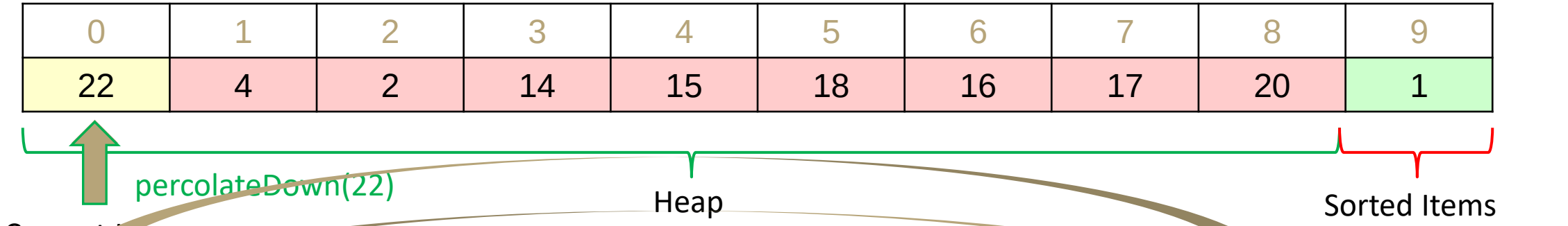
Heapsort – Best Case



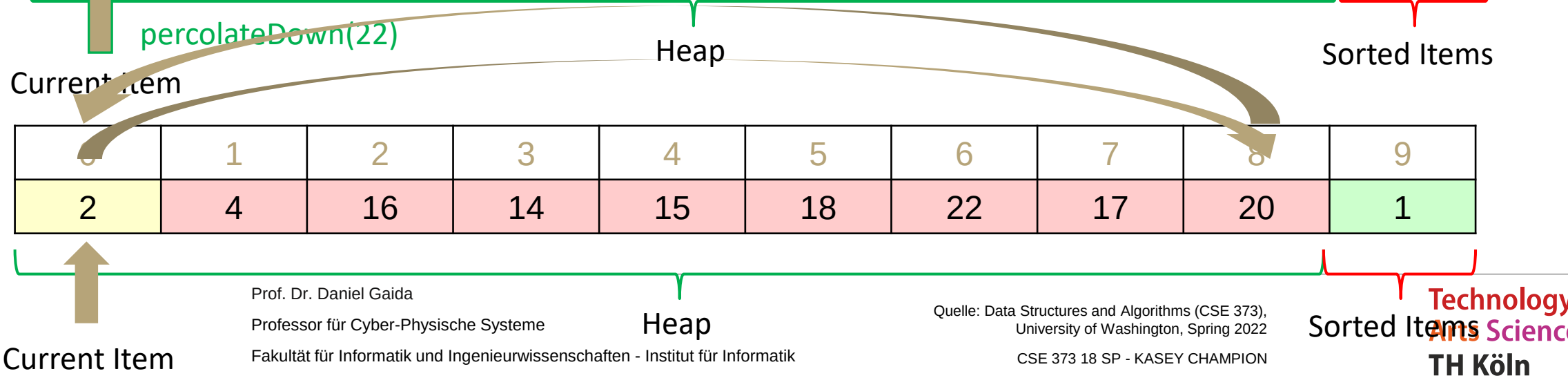
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

In-place Heapsort





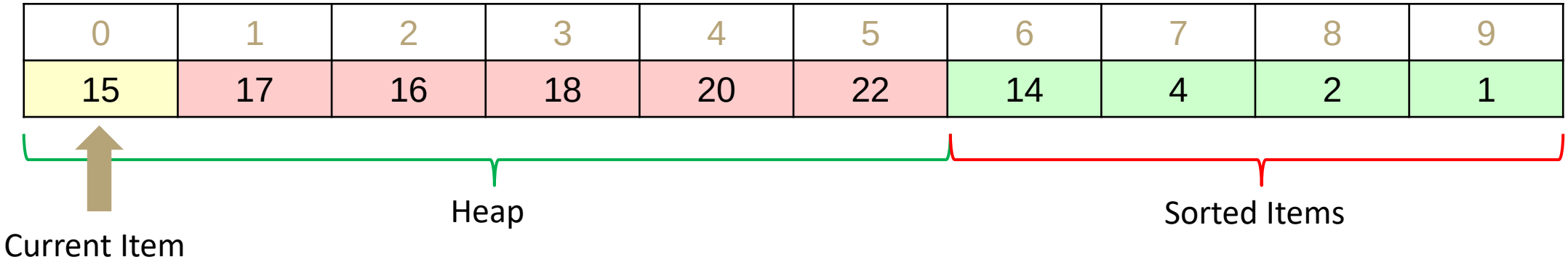
percolateDown(22)



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 18 SP - KASEY CHAMPION

In-place Heapsort




```
public void inPlaceHeapSort(input) {
   buildHeap(input) // alters original array
   for (i = 0; i < n; i++)  // n = input.size()
      heap = input[0:n - i]
      input[n - i - 1] = removeMin(heap)
}
```

Worst case runtime?	$\Theta(n \log n)$
Best case runtime?	$\Theta(n)$
In-practice runtime?	$\Theta(n \log n)$
Stable?	No
In-place?	Yes

Komplikation: Sortierung im finalen Array ist invers!

Mögliche Korrekturen:

- Reverse anschließend ausführen (O(n))
- Einen Max-Heap verwenden

Übersicht soweit

Selectionsort

Worst case runtime?	$\Theta(n^2)$
Best case runtime?	$\Theta(n^2)$
In-practice runtime?	$\Theta(n^2)$
Stable?	No
In-place?	Yes

Insertionsort

Worst case runtime?	$\Theta(n^2)$
Best case runtime?	$\Theta(n)$
In-practice runtime?	$\Theta(n^2)$
Stable?	Yes
In-place?	Yes

Heapsort

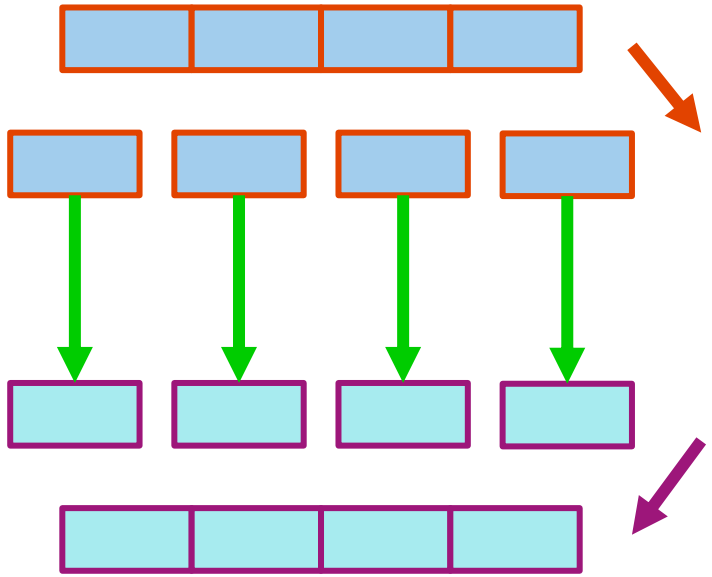
Worst case runtime?	$\Theta(n \log n)$
Best case runtime?	$\Theta(n)$
In-practice runtime?	$\Theta(n \log n)$
Stable?	No
In-place?	Yes

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Algorithmen Entwurfsmuster 3: Divide and Conquer

Allgemeines Rezept:

- Teile die Arbeit rekursiv in kleinere Teile auf
- Lösen der rekursiven Teilprobleme
  - Bei vielen Algorithmen erfordert die Lösung eines Teilproblems keine weitere Bearbeitung, als es rekursiv zu teilen!
- Kombinieren der Ergebnisse der rekursiven Aufrufe



```
divideAndConquer(input) {
  if (small enough to solve):
    conquer, solve, return results
  else:
    divide input into smaller pieces
    recurse on smaller pieces
    combine results and return
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Mergesort

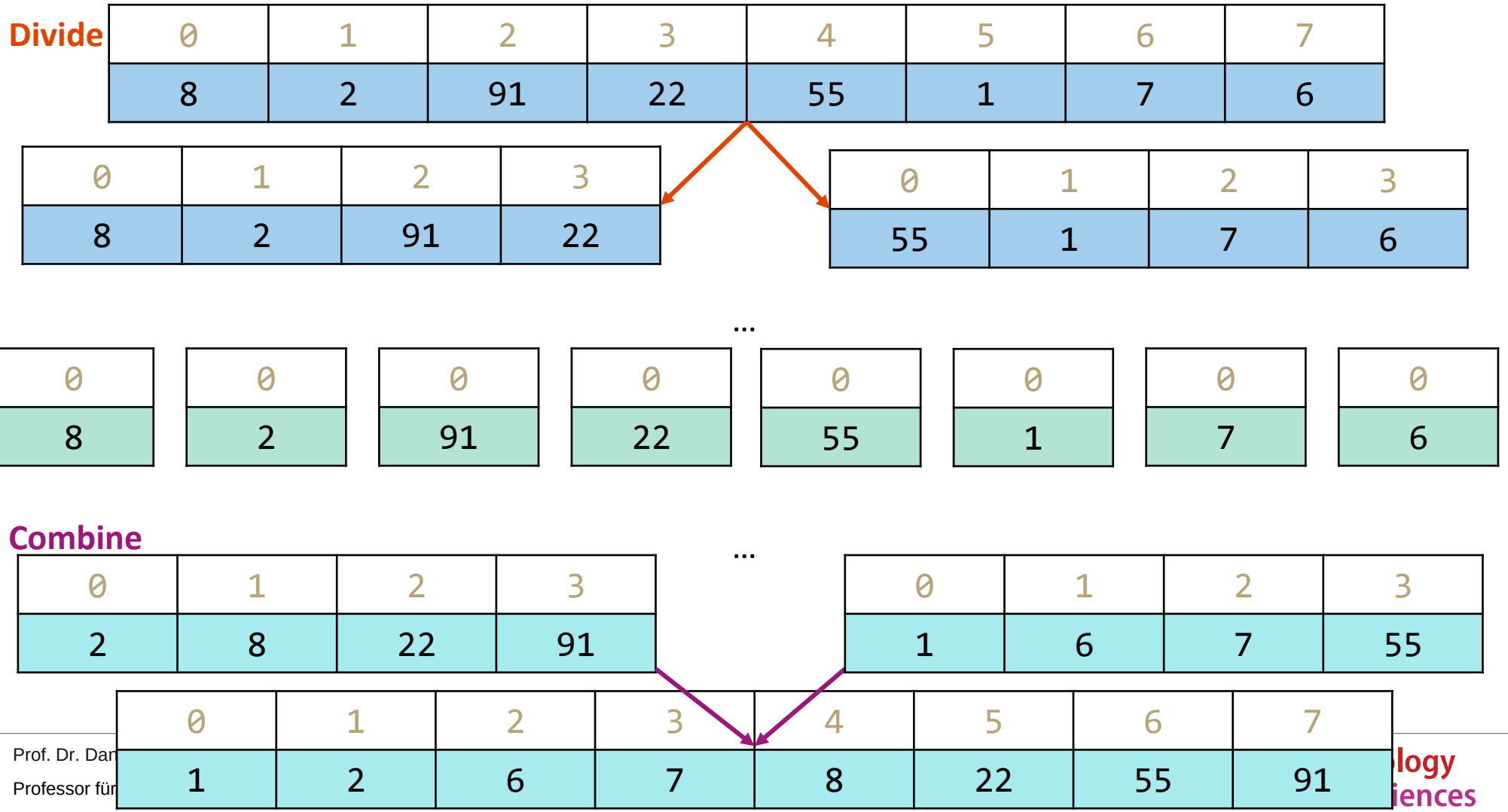
https://www.youtube.com/watch?v=XaqR3G_NVoo

Simply divide in half each time

Conquer

Recursive Calls for each half

The actual sorting happens here!



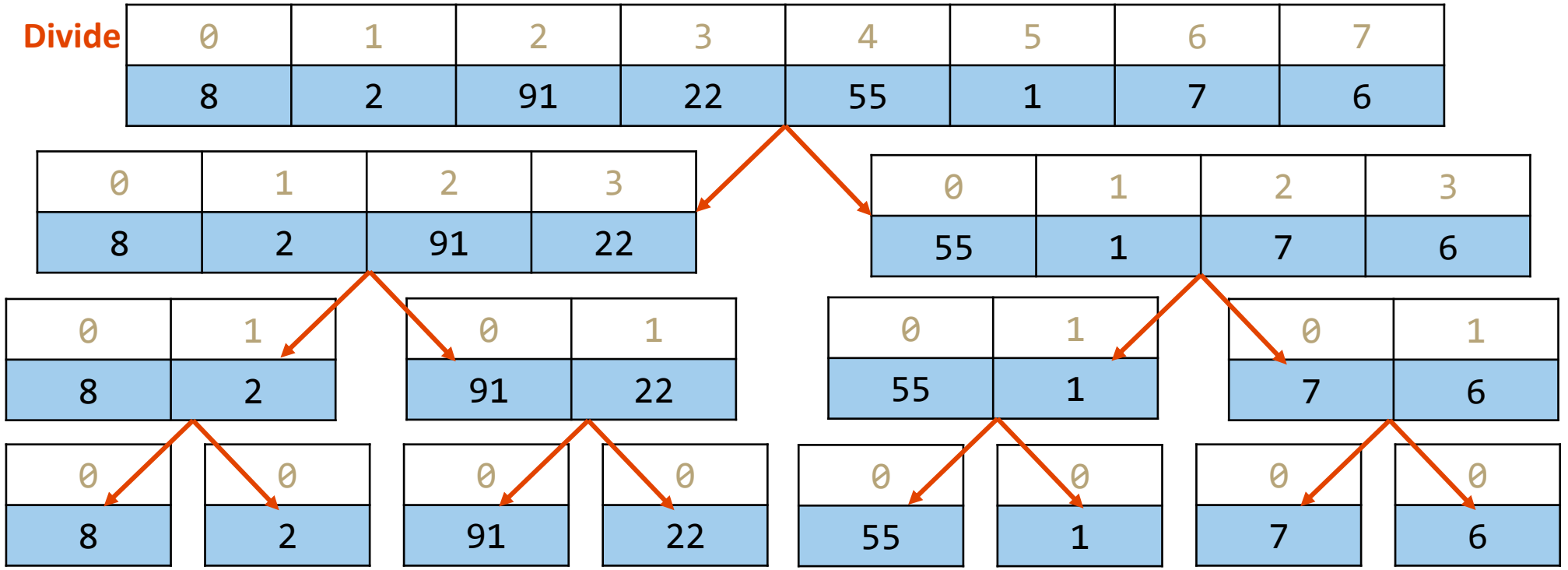
Prof. Dr. Dan
Professor für
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
logy
iences
TH Köln

Mergesort: Divide und Conquer Schritt

Recursive Case: split the array in half

Conquer
and recurse on both halves

Base Case: when array hits size 1, stop dividing.

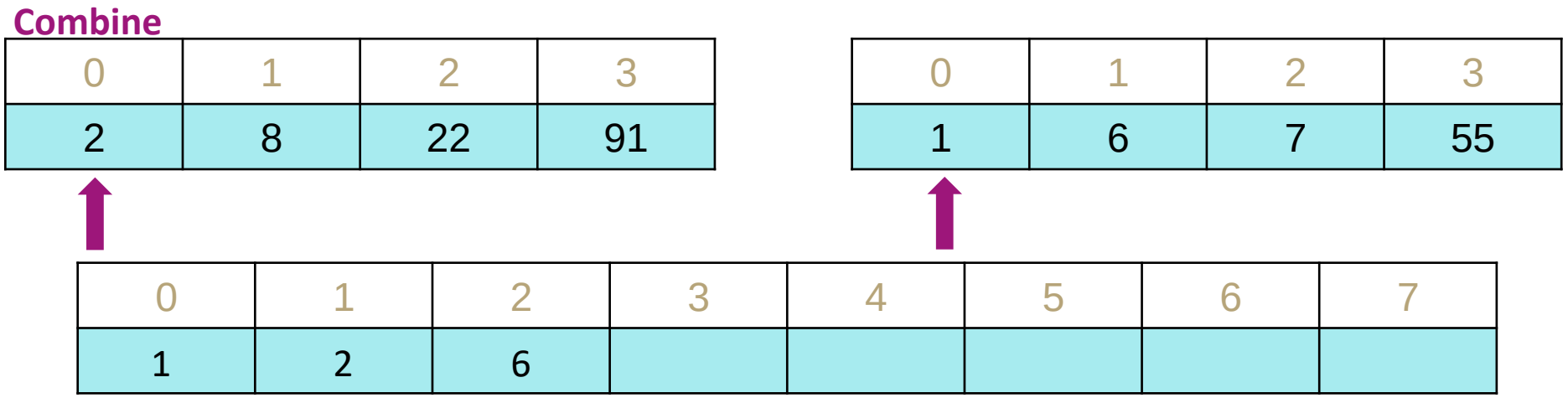


Sort the pieces through the magic of recursion

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

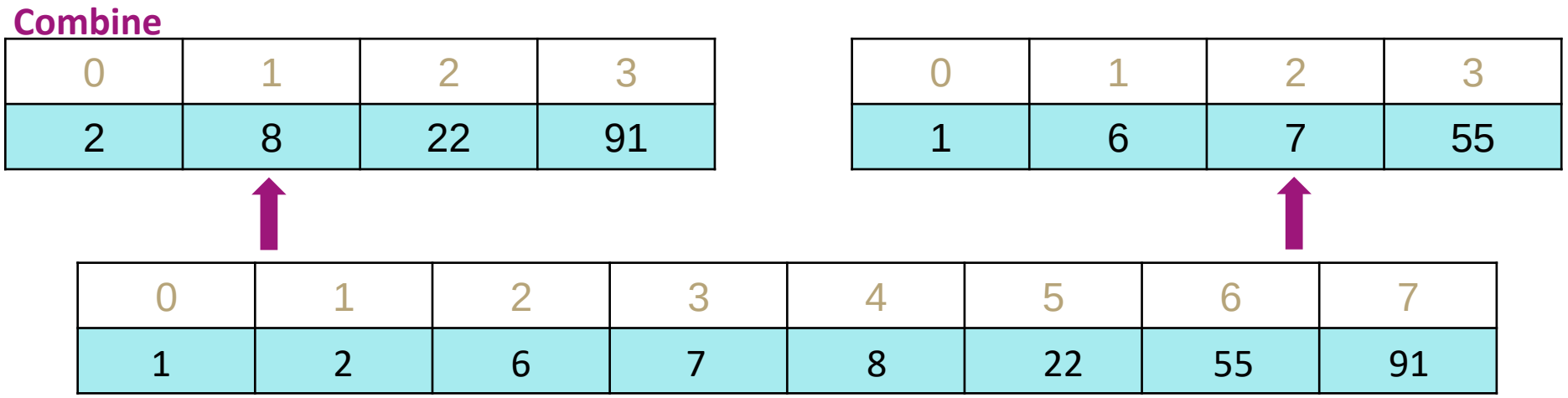
Mergesort: Combine Schritt



Kombinieren von zwei sortierten Arrays:

- Initialisierung von Zeigern auf den Anfang der beiden Arrays
- Wiederholen, bis alle Elemente hinzugefügt sind:
  1. Hinzufügen des kleineren Elements zum Ausgangs-Array
  2. Diesen Zeiger ein Feld weiter schieben

Mergesort: Combine Schritt

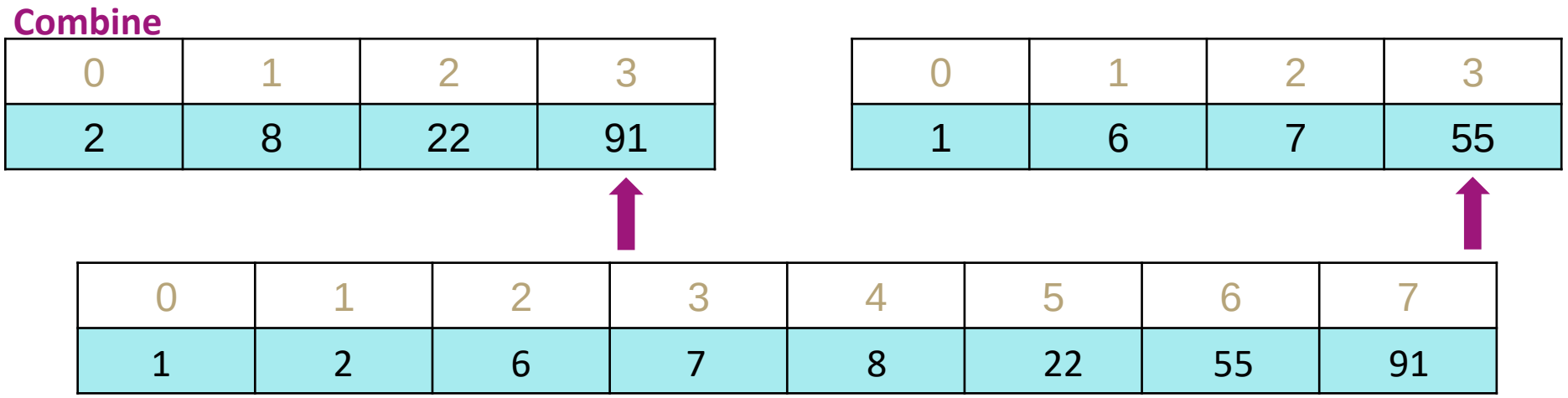


Kombinieren von zwei sortierten Arrays:

- Initialisierung von Zeigern auf den Anfang der beiden Arrays
- Wiederholen, bis alle Elemente hinzugefügt sind:
  1. Hinzufügen des kleineren Elements zum Ausgangs-Array
  2. Diesen Zeiger ein Feld weiter schieben

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Mergesort: Combine Schritt



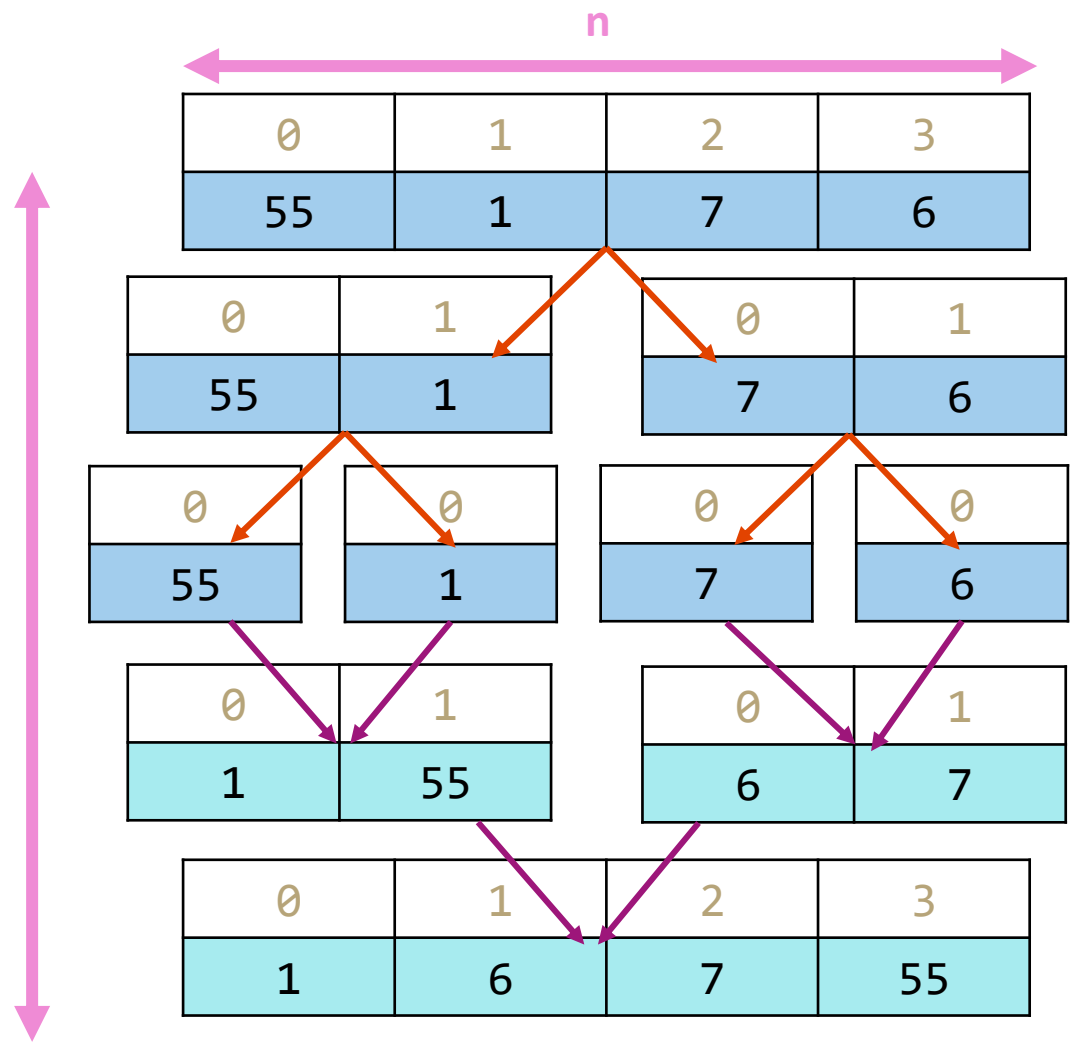
Kombinieren von zwei sortierten Arrays:

- Initialisierung von Zeigern auf den Anfang der beiden Arrays
- Wiederholen, bis alle Elemente hinzugefügt sind:
  1. Hinzufügen des kleineren Elements zum Ausgangs-Array
  2. Diesen Zeiger ein Feld weiter schieben

Das funktioniert, weil wir nur den kleineren Zeiger verschieben - und dann den größeren gegen einen neuen Wert „abwägen“, und weil die Arrays sortiert sind, müssen wir nie zurückgehen!

Mergesort

```
mergeSort(list) {
   if (list.length == 1):
      return list
   else:
      leftHalf = mergeSort(list[0, ..., mid])
      rightHalf = mergeSort(list[mid + 1, ...])
      return merge(leftHalf, rightHalf)
}
```



Worst case runtime? $T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$

Best case runtime? Same

$T(n) \in \Theta(n \log n)$

In Practice runtime? Same

Stable? Yes

Speicherplatzbedarf: O(n log(n))

In-place? No

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Algorithmen Entwurfsmuster 3: Divide and Conquer

Es gibt mehr als eine Art des Divide Schritts!

Mergesort:

- Aufteilung in zwei Arrays.
- Elemente, die sich zufällig auf der linken und auf der rechten Seite befinden.

Quicksort:

- Aufteilung in zwei Arrays.
- Grob gesagt, in Elemente, die „klein“ sind und in Elemente, die „groß“ sind.
- Wie definiert man „klein“ und „groß“? Wählen Sie ein Pivotelement im Array, das die beiden Arrays teilt!

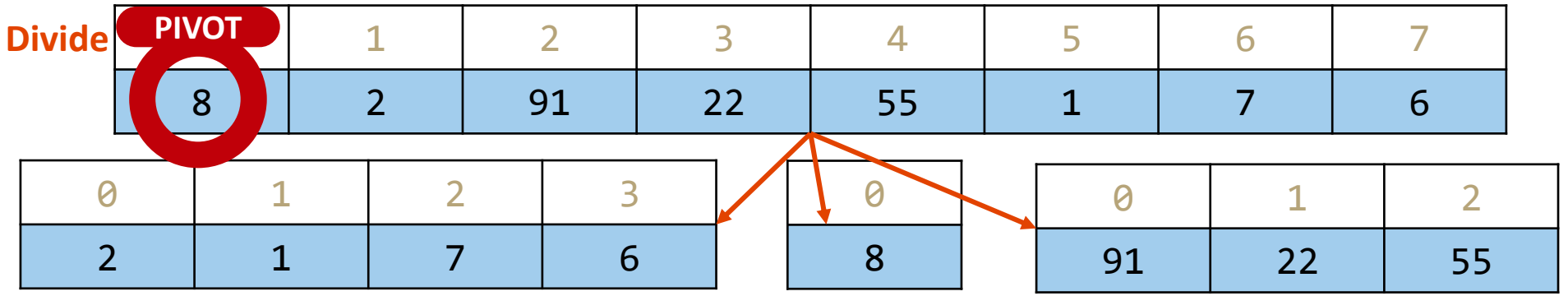
- „Das Pivotelement (franz. pivot ‚Dreh-, Angelpunkt‘) ist dasjenige Element einer Zahlenmenge, das als Erstes von einem Algorithmus ausgewählt wird“ (wikipedia)

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

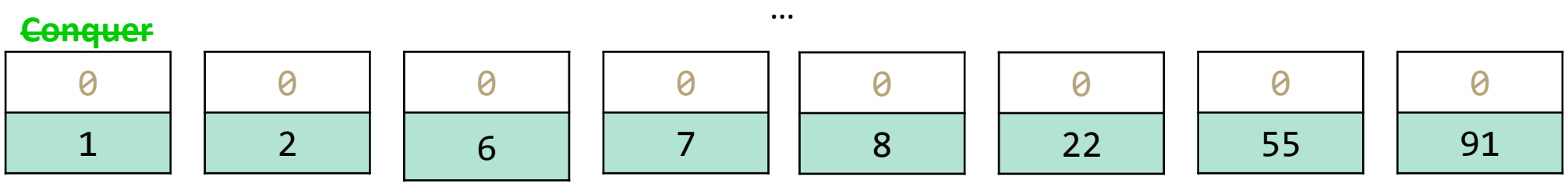
Quicksort (v1)

https://www.youtube.com/watch?v=ywWBy6J5gz8

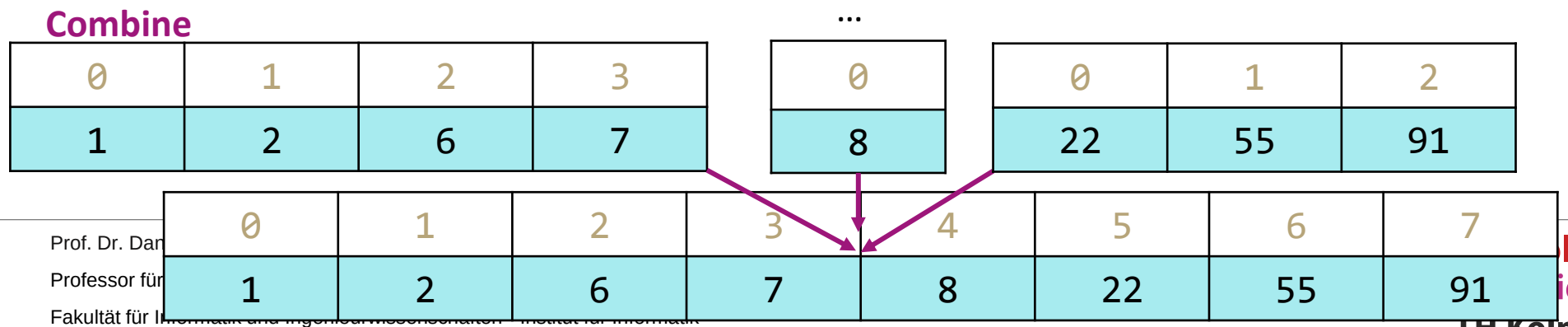
Choose a “pivot” element, partition array relative to it!



No extra conquer step needed!



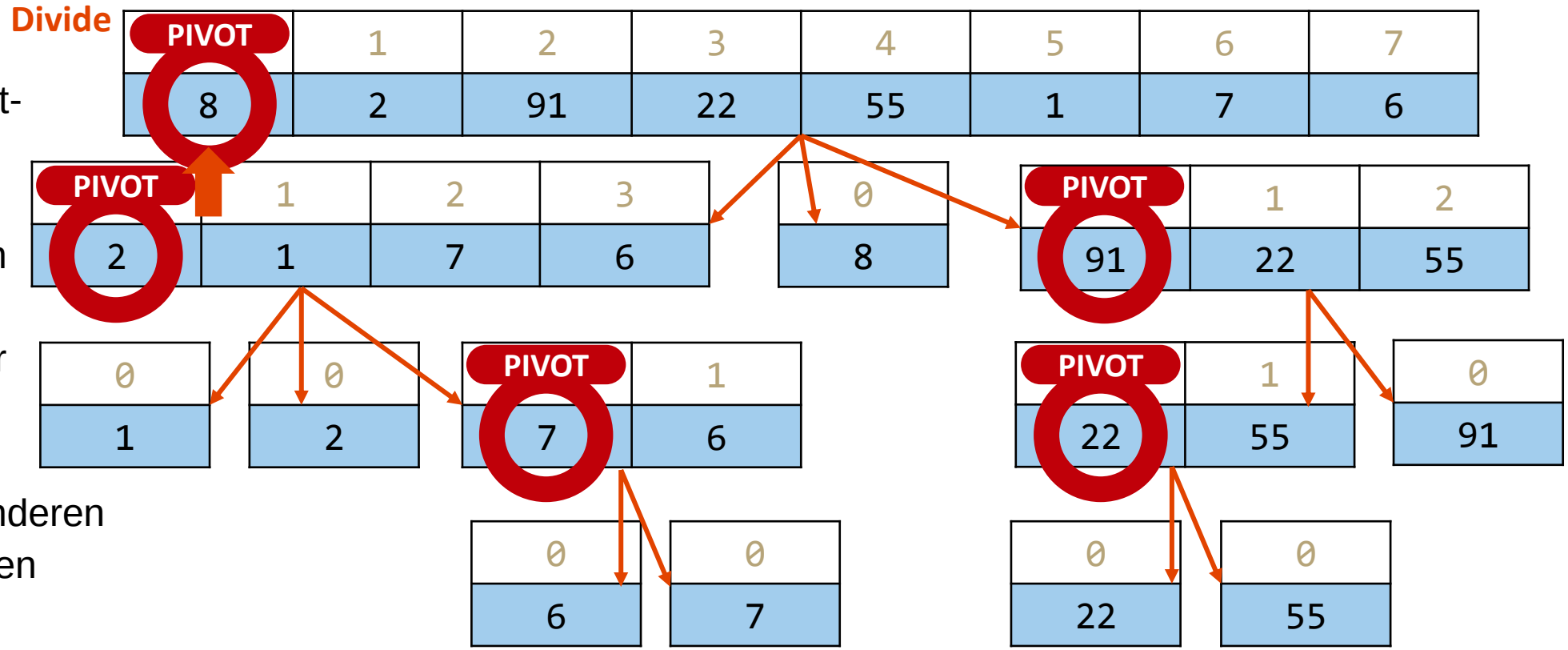
Simply concatenate the now-sorted arrays!



Quicksort (v1): Divide Schritt

Rekursiver Fall:

- Wählen Sie ein Pivot-element
- Partitionierung: lineares Durchlaufen des Arrays, Hinzufügen kleinerer Elemente zu einem Array und größerer Elemente zu dem anderen
- Rekursiv partitionieren

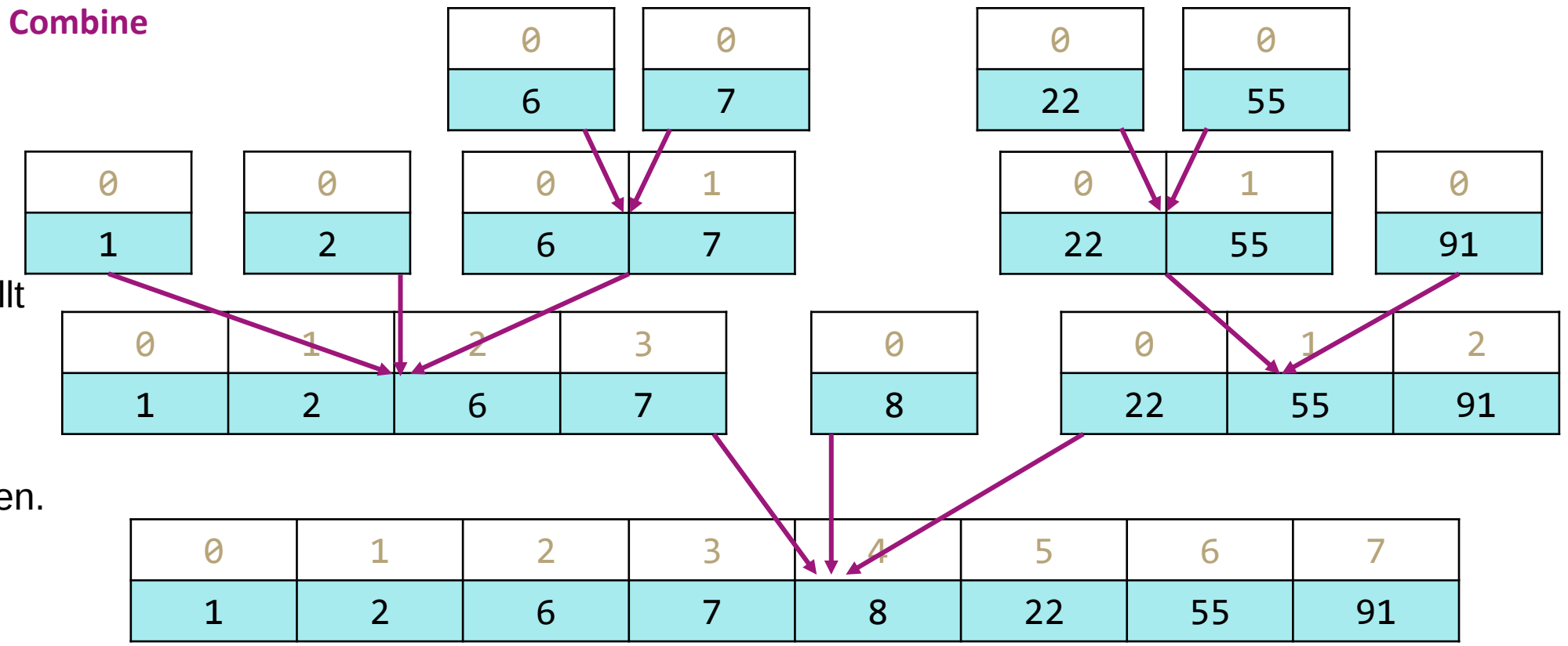


Basisfall:

- Wenn Array die Größe 1 erreicht, Partitionierung beenden.

Quicksort (v1): Combine Schritt

Konkatenieren Sie die Arrays, die zuvor erstellt wurden! Der Partitionsschritt hat sie bereits in der richtigen Reihenfolge hinterlassen.



Quicksort (v1)

```
quickSort(list) {
   if (list.length <= 1):
      return list
   else:
      pivot = choosePivot(list)
      smallerHalf = quickSort(getSmaller(pivot, list))
      largerHalf = quickSort(getBigger(pivot, list))
      return smallerHalf + pivot + largerHalf
}
```

O(n)

Worst case runtime? $T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ T(n-1) + n & \text{sonst} \end{cases}$

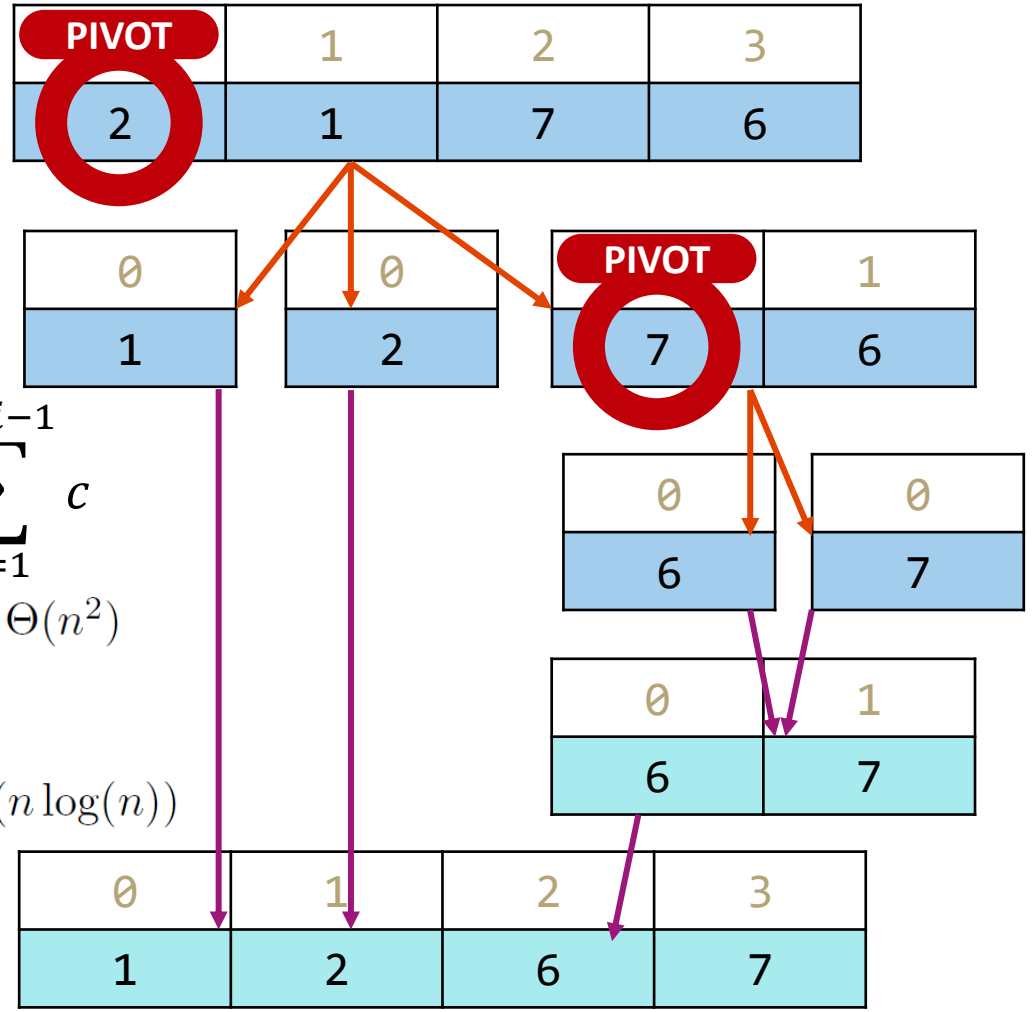
Best case runtime? $T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{sonst} \end{cases}$

In-practice runtime? **Just trust me: $\Theta(n \log n)$**
(absurd amount of math to get here)

Stable? No

In-place? Can be done!

Worst case: Pivot only chops off one value
Best case: Pivot divides each array in half



$$\sum_{i=0}^{n-2} \sum_{j=1}^{n-i-1} c$$

$T(n) \in \Theta(n^2)$

$T(n) \in \Theta(n \log(n))$

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Quicksort (v1) – Worst Case

```
quickSort(list) {
   if (list.length <= 1):
      return list
   else:
      pivot = choosePivot(list)
      smallerHalf = quickSort(getSmaller(pivot, list))
      largerHalf = quickSort(getBigger(pivot, list))
      return smallerHalf + pivot + largerHalf
}
```

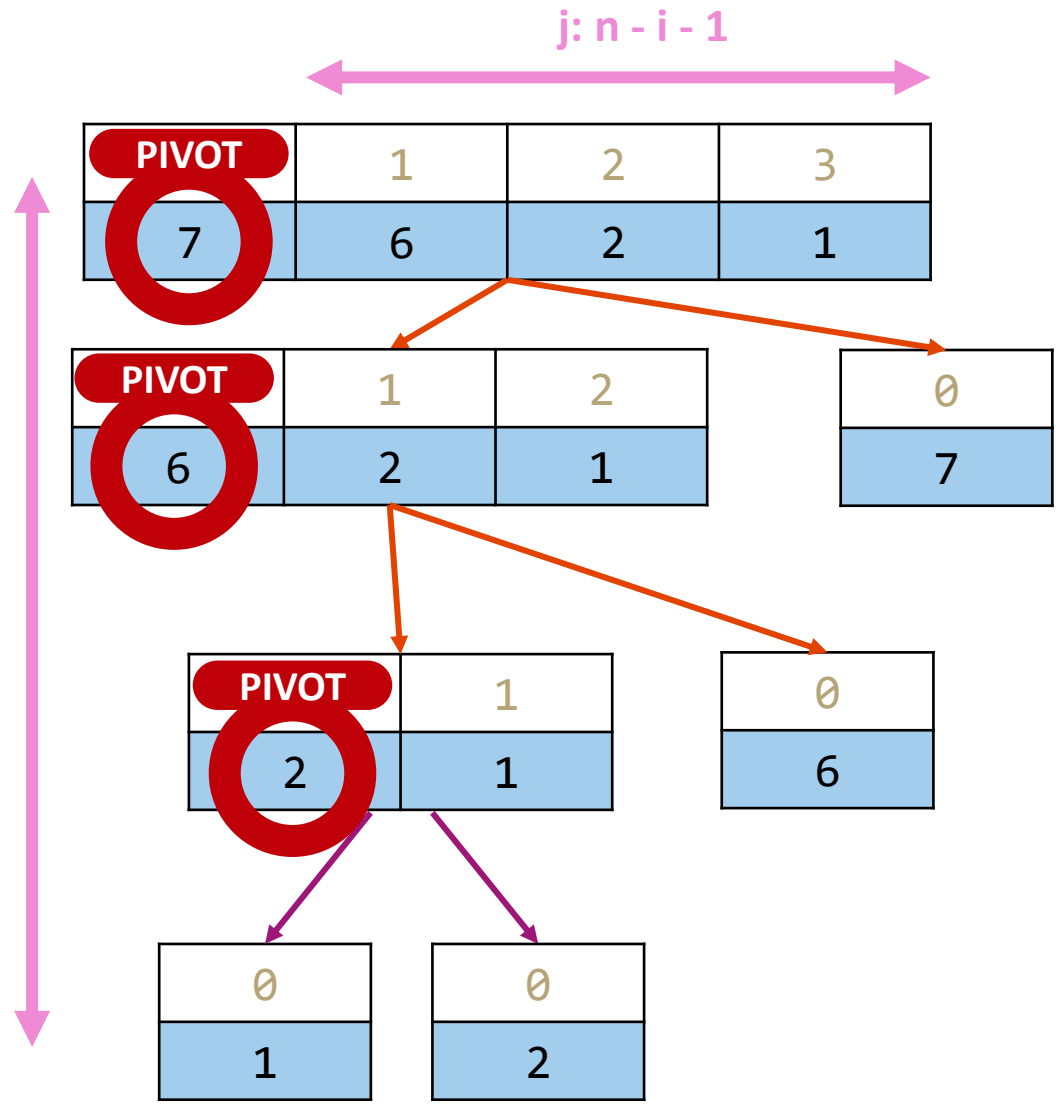
Worst case runtime? $T(n) = \begin{cases} 1 & \text{falls } n \leq 1 \\ T(n-1) + n & \text{sonst} \end{cases}$

$$\sum_{i=0}^{n-2} \sum_{j=1}^{n-i-1} c$$

getSmaller

$T(n) \in \Theta(n^2)$

Worst case: Pivot only chops off one value



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle der Bilder: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Quicksort: Vermeidung des Worst Case

Wie lässt sich der Worst Case vermeiden?

- Es kommt alles auf das Pivotelement an. Wenn das Pivotelement jedes Array in zwei Hälften teilt, ergibt sich ein besseres Verhalten

Quicksort: Strategien um das Pivotelement zu bestimmen

Einfach das erste Element als Pivotelement nehmen

- Sehr schnell
- Worst Case: Quicksort hat Laufzeit von $\Omega(n^2)$

Nehmen Sie den Median aus dem ersten, letzten und mittleren Element

- Macht das Pivotelement ein bisschen von Arrayelemente abhängig, zumindest wird nicht das größte/kleinste Element gewählt
- Worst Case ist auch $\Omega(n^2)$ (keine bestätigte Quelle), auf Daten der realen Welt funktioniert Algorithmus sehr gut!

Wähle ein zufälliges Element als Pivotelement

- Laufzeit in $O(n \log n)$ mit Wahrscheinlichkeit von mindestens $1 - 1/n^2$ (keine bestätigte Quelle für Zahl)
- Es gibt keinen einfachen Worst Case Datensatz (bspw. sortiert, umgekehrt sortiert)

Sortieralgorithmen: Übersicht

	Best Case	Worst Case	Space	Stable
Selectionsort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	No
Insertionsort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	Yes
Heapsort	$\Theta(n)$	$\Theta(n\log n)$	$\Theta(n)$	No
In-Place Heapsort	$\Theta(n)$	$\Theta(n\log n)$	$\Theta(1)$	No
Mergesort	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$ $\Theta(n)$* optimized	Yes
Quicksort	$\Theta(n\log n)$	$\Theta(n^2)$	$\Theta(n)$	No
In-place Quicksort	$\Theta(n\log n)$	$\Theta(n^2)$	$\Theta(1)$	No

* They actually use Tim Sort, which is very similar to Merge Sort in theory, but has some minor details different

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Sortieralgorithmen: Zusammenfassung

What does Java do?

- Actually uses a combination of 3 different sorts:
  - If objects: use Merge Sort* (stable!)
  - If primitives: use Dual Pivot Quick Sort
  - If “reasonably short” array of primitives: use Insertion Sort
    - Researchers say 48 elements

Wichtigste Erkenntnis: Kein einzelner Sortieralgorithmus ist „der Beste“!

- Verschiedene Sortieralgorithmen haben unterschiedliche Eigenschaften in verschiedenen Situationen
- Der „beste“ Sortieralgorithmus ist derjenige, die für Ihre Daten am besten geeignet ist.

* They actually use Tim Sort, which is very similar to Merge Sort in theory, but has some minor details different

STRATEGY 1:
ITERATIVE IMPROVEMENT

STRATEGY 2:
IMPOSE STRUCTURE

STRATEGY 3:
DIVIDE AND CONQUER

Insertion Sort

WORST

BEST

Simple, stable, low-overhead, great if already sorted.

IN-PLACE STABLE SPACE

Selection Sort

WORST

BEST

Minimizes array writes, otherwise never preferred.

IN-PLACE SPACE

Heap Sort

WORST

BEST

Always good runtimes

IN-PLACE SPACE

Merge Sort

WORST

BEST

Stable, very reliable! In-place variant is slower.

STABLE SPACE

Quick Sort

WORST

BEST

Fastest in practice (constant factors), bad worst case.

IN-PLACE SPACE

Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Arts Sciences
TH Köln

Sortieralgorithmen: Zusammenfassung

Bisher haben wir etwas über vergleichsbasiertes Sortieren gelernt

- Funktionieren bei jedem vergleichbaren Objekt
- Haben im Worst Case eine untere Schranke von Ω(n log n)

Das liegt daran, dass beim Sortieren mit Vergleichen alle Elemente miteinander verglichen werden müssen

Heapsort:

- O(n) Laufzeit, um alle Werte in eine geordnete Struktur (Baum) zu bringen
- O(log n) Laufzeit zum Entfernen von Elementen aus der Struktur in sortierter Reihenfolge

Nicht-vergleichsbasierte Sortieralgorithmen

Kann die Laufzeit im Worst Case besser sein als Ω(n log n)?

- Für vergleichsbasiertes Sortieren: NEIN. Eine bewiesene untere Schranke!
  - Intuition: n Elemente zu sortieren, kein schnellerer Weg, den „richtigen Platz“ zu finden als log n
- Nicht-vergleichsbasierte Sortieralgorithmen können jedoch in bestimmten Situationen besser sein!

Beispiele für nicht-vergleichsbasierte Sortieralgorithmen:

- Radix Sort (Wikipedia, VisuAlgo)
- Counting Sort (Wikipedia)
- Bucketsort (Wikipedia)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Bucketsort (aka Bin Sort)

Wenn zu sortierende Elemente ganze Zahlen sind, die im Wertebereich von 1 bis K liegen

- Array der Größe K erstellen und jedes Element in den entsprechenden Behälter legen („scatter“)
- Hier einfach die Anzahl der ganzen Zahlen in jedem Behälter speichern
- Ausgabe der Ergebnisse durch lineares Durchlaufen des Arrays der Behälter („gather“)

elements:

[5, 1, 3, 4, 3, 2, 1, 1, 5, 4, 5]

O(n)

arr:

Anzahl 1er in elements

0	3
1	1
2	2
3	2
4	3

O(K + n)

[1, 1, 1, 2, 3, 3, 4, 4, 5, 5, 5]

```
for el in elements:
  arr[el - 1] += 1
```

Total Runtime: O(K + n)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

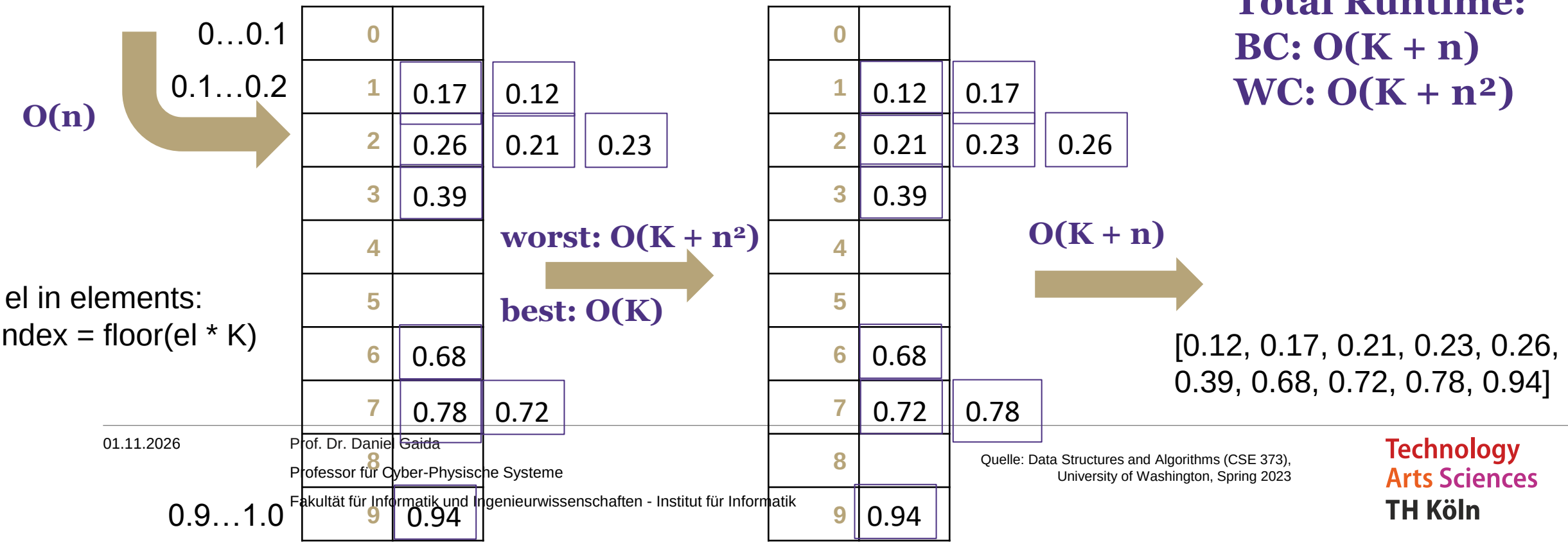
Bucketsort für Float-Werte in [0, 1]

[Bucket Sort Example Video]

- K Behälter als eine Liste von Float-Werten erstellen
- Float-Werte in die Behälter legen, mit Insertionsort einzelne Behälter sortieren

elements:
[0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]

for el in elements:
index = floor(el * K)



Total Runtime:
BC: O(K + n)
WC: O(K + n²)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bucketsort

```
function bucketSort(array, K) is
    buckets ← new array of K empty lists
    M ← 1 + the maximum key value in the array
    for i = 0 to length(array) do
        insert array[i] into buckets[floor(K × array[i] / M)]
    for i = 0 to K - 1 do
        nextSort(buckets[i])
    return the concatenation of buckets[0], ..., buckets[K-1]
```

Worst case runtime?	O(K + n) for ints O(K + n^2) for float if insertion sort is used
Best case runtime?	O(n) if K ≤ n, for ints, and for floats, K = n, if in each bucket there is only one float
In-practice runtime?	O(n) if K ≤ n, for ints, and for floats, K ≅ n, if values are evenly distributed
Stable?	Can be because of insertion sort
In-place?	No
Useful for:	Integer: When range, K, is smaller or not much larger than n (many duplicates) Not good when K >> n, wasted space Floats: evenly distributed

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Sortieralgorithmen: Übung

Wenn ich Ihnen einen Stapel Veröffentlichungen (Paper) gebe und Sie bitten würde, sie alphabetisch nach Autorennamen zu sortieren, wie würden Sie dies tun?

Selection Sort - I would flip through the stack from front to back looking for the first name, then pull it to the front. Then I would flip through again looking for the second name and put it behind the first and so on until all were sorted

Insertion Sort - I look at the first two papers and put them in sorted order, then look at the third and put it in sorted order with the previous two and continue until the whole stack is in sorted order

Merge Sort - I would spread the papers out on the ground and break them into subsections, sort the subsections section by section then put them all back together

Bucket Sort - I would start by putting the papers into groups based on the first letter of the author's name until I had 26 piles, then I would sort within those piles and put them all back together

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bucketsort - Übung

Apply Bucket Sort to the sequence

[7625, 5002, 6746, 7403, 2266, 5532, 2010]

1. scatter to buckets
2. perform insertion on each bucket
3. gather buckets:
   [2010, 2266, 5002, 5531, 6746, 7625, 7403]

0	
1	
2	2266, 2010
3	
4	
5	5002, 5532
6	6746
7	7625, 7403
8	
9	

0	
1	
2	2010, 2266
3	
4	
5	5002, 5531
6	6746
7	7625, 7403
8	
9	

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Nächste Vorlesung und Fragen

- Dynamische Programmierung
- Fragen?

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik